

FPGA RACING GAME

1. Pendahuluan

Pada dokumen ini, akan dibuat sebuah game mobil menghindari rintangan yang berjalan sepenuhnya di dalam FPGA menggunakan Verilog HDL. Pemain mengendalikan sebuah mobil yang ditampilkan pada monitor VGA, menghindari rintangan yang datang, sambil skor terus bertambah. Game dilengkapi layar start dengan hitung mundur, kondisi game over, serta efek suara dan musik.

Instalasi software yang dibutuhkan:

- Quartus Prime Lite Edition (gratis) v25.1 [\[Link\]](#)

Hardware yang digunakan:

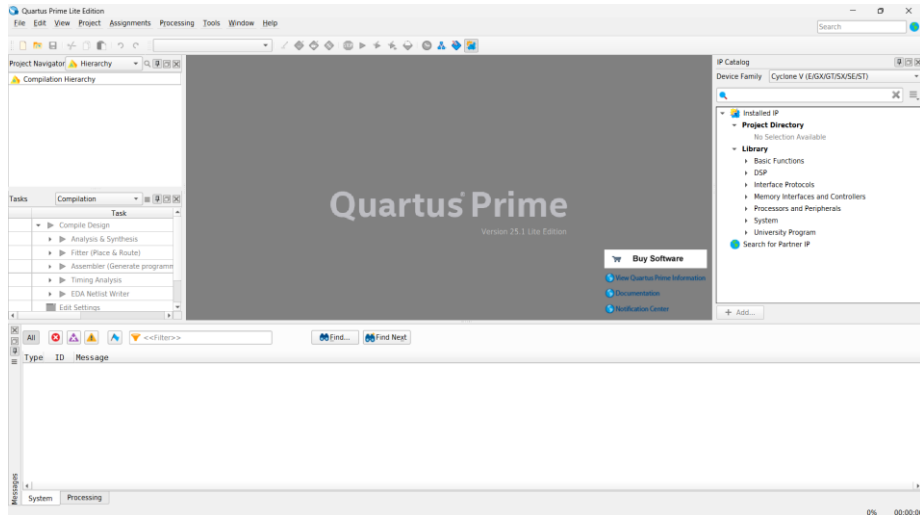
Barang	Keterangan
Board DE10-Standard	Cyclone V 5CSXFC6D6F31C6, beserta adaptor power
Kabel USB	Untuk memprogram lewat USB-Blaster II
Monitor + kabel VGA	Wajib memiliki port VGA (bukan hanya HDMI)

Panduan ini memiliki 7 tahap:

1. Membuat Proyek Quartus
2. Membuat File VGA 640x480
3. Menyusun Kerangka Top-Level
4. Rencana Layout Layar
5. Membuat File Render
6. Menyambungkan render_static ke car_game
7. Kompilasi, Pengaturan Pin, dan Uji Coba di Board FPGA
8. Insigh

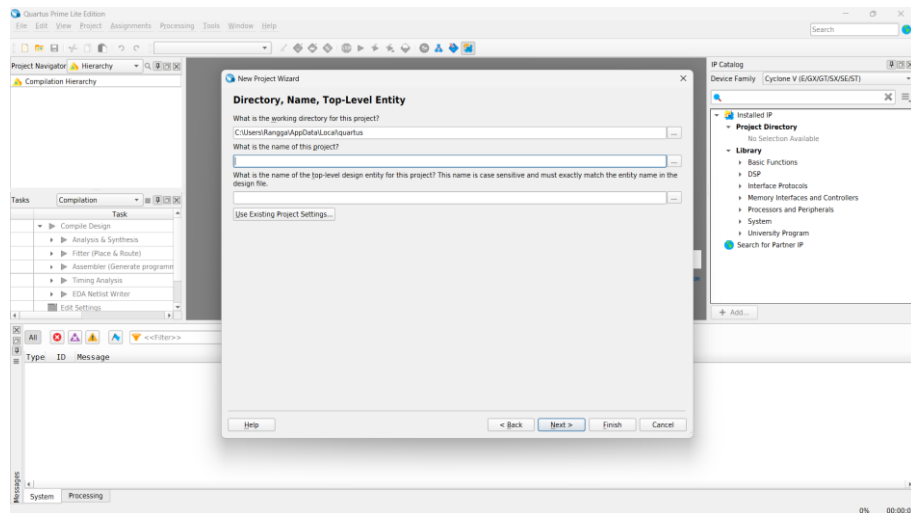
2. Membuat Proyek Quartus Baru

- a. Buka Quartus Prime Lite v25.1.



b. Buat Proyek baru: **File >> New Project Wizard...** >> klik **Next**

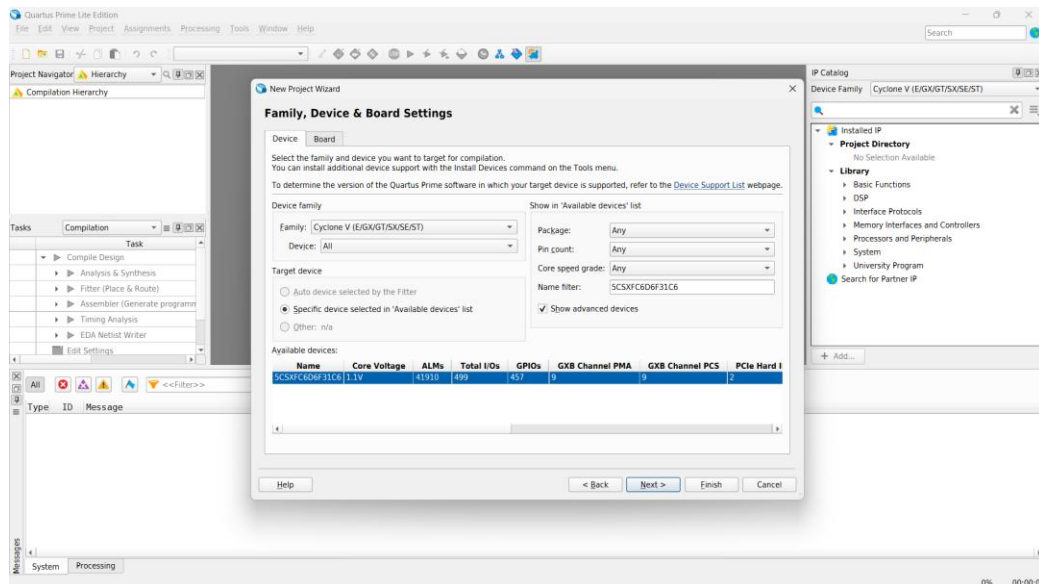
c. Isi kotak yang tersedia lalu klik **Next**:



d. Project Type: **Empty Project** >> **Next**.

e. Add files: **Empty** >> kosongkan dulu, klik **Next**

f. Family, Device & Board Settings: atur **Family = Cyclone V (E/GX/GT/SX/SE/ST)**, lalu pada **Name Filter** ketik **5CSXFC6D6F31C6** dan pilih device tersebut di daftar >> **Next**.

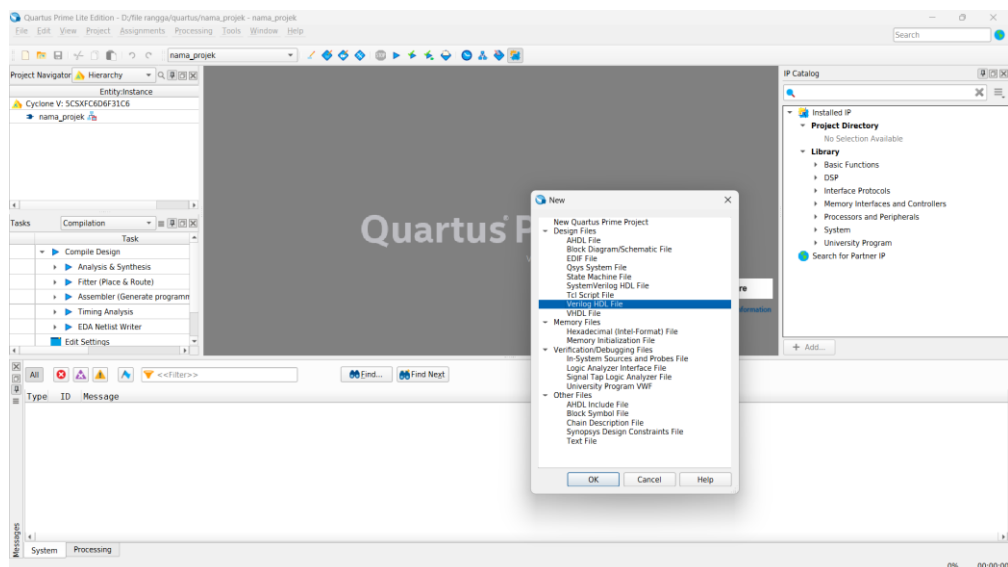


g. EDA Tool Setting >> Next.

h. Summary >> klik Finish.

3. Membuat File VGA 640x480

A. File >> New >> Design Files >> Verilog HDL File >> OK.



B. Copy paste potongan kode berikut ke layar text-editor Quartus:

vga_640x480

```
module vga_640x480 (
    input wire pixel_clk, // detak 25 MHz, 1 pixel per detak
    input wire reset,      // reset synchronous, aktif-high
    output reg [9:0] x,     // kolom pixel yang sedang digambar
    output reg [9:0] y,     // baris pixel yang sedang digambar
    output wire video_on,  // 1 = pixel ini terlihat mata
    output wire hsync,     // pulsa sinkronisasi horizontal
    output wire vsync,     // pulsa sinkronisasi vertikal
);
```

```

// Kenapa x,y bertipe REG tapi video_on,hsync,vsync bertipe WIRE?
// - reg = nilainya "diingat" antar-detak, hanya berubah di dalam
//       blok always (seperti variabel yang punya memori).
// - wire = nilainya dihitung ULANG terus-menerus dari sinyal lain,
//       tidak punya memori sendiri (lewat kata kunci assign).

localparam H_VISIBLE = 640;
localparam H_FRONT   = 16;
localparam H_SYNC    = 96;
localparam H_BACK     = 48;
localparam H_TOTAL   = 800;

localparam V_VISIBLE = 480;
localparam V_FRONT   = 10;
localparam V_SYNC    = 2;
localparam V_BACK     = 33;
localparam V_TOTAL   = 525;

// always @(posedge pixel_clk): blok ini HANYA jalan tepat saat
// pixel_clk berpindah dari 0 ke 1 (rising edge) bukan terus-
// menerus. Ini yang membuat x,y berjalan selaras dengan detak.
always @(posedge pixel_clk) begin
    if (reset) begin
        x <= 10'd0; // <= artinya "nanti diisi" (non-blocking),
        y <= 10'd0; // dipakai KHUSUS untuk reg di dalam always
    end else begin
        if (x == H_TOTAL - 1) begin
            x <= 10'd0;
            if (y == V_TOTAL - 1)
                y <= 10'd0;
            else
                y <= y + 10'd1;
        end else begin
            x <= x + 10'd1;
        end
    end
end

// assign dipakai untuk WIRE: nilainya dihitung terus-menerus
// (kombinasi), langsung mengikuti x,y saat itu juga
// beda dengan reg yang cuma berubah saat detak di always.
assign video_on = (x < H_VISIBLE) && (y < V_VISIBLE);
assign hsync = ~(x >= H_VISIBLE + H_FRONT) &&
               (x < H_VISIBLE + H_FRONT + H_SYNC));
assign vsync = ~(y >= V_VISIBLE + V_FRONT) &&
               (y < V_VISIBLE + V_FRONT + V_SYNC));
endmodule

```

C. Simpan sebagai **vga_640x480.v** : **File >> Save As... >> vga_640x480.v >> Save**

Saat menyimpan file baru, Quartus biasanya menampilkan kotak centang "Add file to current project" pastikan tercentang. Kalau terlewat, file tidak masuk ke daftar proyek dan nanti muncul error "module not found" saat kompilasi, meski kodenya sendiri sudah benar.

4. Menyusun Kerangka Top-Level

A. Buat file baru (**File >> New >> Design Files >> Verilog HDL File**)

B. Copy paste potongan berikut:**car_game.v (kerangka awal)**

```

module car_game (
    input wire      CLOCK_50,
    input wire [4:0] KEY,    // belum dipakai di tahap ini akan
                             // disambung untuk kendali kemudi (PS/2)
                             // pada tahap pengembangan lanjutan
    input wire [9:0] SW,

    output wire      VGA_CLK,
    output wire      VGA_HS,
    output wire      VGA_VS,
    output wire      VGA_BLANK_N,
    output wire      VGA_SYNC_N,
    output wire [7:0] VGA_R,
    output wire [7:0] VGA_G,
    output wire [7:0] VGA_B
);
    reg pixel_clk = 1'b0;
    always @(posedge CLOCK_50) begin
        pixel_clk <= ~pixel_clk; // dibagi 2 -> 25 MHz
    end

    wire reset;
    wire video_on;
    wire [9:0] x;
    wire [9:0] y;

    assign reset = SW[9];

    vga_640x480 vga_unit (
        .pixel_clk (pixel_clk),
        .reset      (reset),
        .x          (x),
        .y          (y),
        .video_on   (video_on),
        .hsync      (VGA_HS),
        .vsync      (VGA_VS)
    );

    assign VGA_R = 8'h00;
    assign VGA_G = 8'h00;
    assign VGA_B = 8'h00;

    assign VGA_CLK = pixel_clk;
    assign VGA_BLANK_N = video_on;
    assign VGA_SYNC_N = 1'b1;
endmodule

```

C. Simpan sebagai `car_game.v`. Modul ini menjadi top-level entity proyek (ditetapkan lengkap di tahap kompilasi).**5. Rencana Layout Layar**

Layar 640x480 dibagi jadi dua daerah oleh satu baris horizon (HORIZON_Y). Semua baris di atasnya adalah langit; semua baris di bawahnya adalah tanah, tempat jalan dan rumput digambar. Supaya jalan terlihat menjauh (efek perspektif), lebar jalan tidak dibuat tetap: sempit di horizon, melebar setiap baris turun mendekati bagian bawah layar.

6. Membuat File Render

A. Buat file baru (**File >> New >> Design Files >> Verilog HDL File**)

B. Copy paste potongan berikut:

```
render_static.v

module render_static (
    input wire      clk,          // belum dipakai disiapkan untuk
                                   // animasi scrolling pada tahap lanjutan
    input wire      video_on,    // 1 = pixel ini terlihat mata
    input wire [9:0] x,          // kolom pixel yang sedang digambar
    input wire [9:0] y,          // baris pixel yang sedang digambar
    output reg [7:0] red,         // warna merah untuk pixel ini, 0-255
    output reg [7:0] green,      // warna hijau untuk pixel ini, 0-255
    output reg [7:0] blue        // warna biru untuk pixel ini, 0-255
);
    localparam HORIZON_Y      = 10'd120; // baris tempat horizon berada
    localparam ROAD_CENTER_X = 10'd320; // tengah jalan = tengah layar
    localparam ROAD_HALF_FAR = 10'd20;  // separuh lebar jalan di horizon

    // dy = seberapa jauh baris y ini dari horizon (0 kalau masih langit)
    wire [9:0] dy = (y >= HORIZON_Y) ? (y - HORIZON_Y) : 10'd0;

    // makin besar dy (makin turun/dekat), makin lebar jalannya
    // inilah yang menciptakan efek perspektif/menjauh
    wire [9:0] road_half = ROAD_HALF_FAR + (dy >> 1);
    wire [9:0] road_left  = ROAD_CENTER_X - road_half; // tepi kiri jalan
    wire [9:0] road_right = ROAD_CENTER_X + road_half; // tepi kanan jalan

    // road_area = 1 kalau pixel (x,y) ini ada di dalam badan jalan
    wire road_area = (y >= HORIZON_Y)
        && (x >= road_left) && (x < road_right);
    // sky_area = 1 kalau pixel ini ada di atas horizon (langit)
    wire sky_area = (y < HORIZON_Y);

    localparam CAR_CENTER_X = ROAD_CENTER_X; // mobil di tengah jalan
    localparam CAR_WIDTH    = 10'd40;        // lebar kotak mobil
    localparam CAR_TOP_Y    = 10'd400;        // posisi atas kotak mobil
    localparam CAR_HEIGHT   = 10'd60;        // tinggi kotak mobil

    wire [9:0] car_left  = CAR_CENTER_X - (CAR_WIDTH >> 1); // tepi kiri
    wire [9:0] car_right = CAR_CENTER_X + (CAR_WIDTH >> 1); // tepi kanan
    // car_area = 1 kalau pixel (x,y) ini ada di dalam kotak mobil
    wire car_area = (x >= car_left) && (x < car_right) &&
        (y >= CAR_TOP_Y) && (y < CAR_TOP_Y + CAR_HEIGHT);

    // always @(*): beda dengan always @(posedge ...) di vga_640x480.v
    // blok ini TIDAK menunggu detak, langsung dihitung ulang setiap kali
    // salah satu sinyal di dalamnya berubah (murni kombinasional, cocok
    // untuk red/green/blue yang harus selalu "siap" tiap saat).
```

```

always @(*) begin
    // Perhatikan urutan if/else if di bawah ini PENTING:
    // kondisi paling ATAS "menang" duluan kalau beberapa area
    // saling tumpang tindih. Karena itu car_area dicek PALING
    // AWAL (mobil harus tampak di depan/menimpa jalan & langit).
    if (!video_on) begin
        // = di sini artinya "langsung" (blocking) dipakai
        // karena kita di dalam always @(*), bukan always @(posedge)
        red = 8'd0; green = 8'd0; blue = 8'd0; // di luar layar: hitam
    end else if (car_area) begin
        red = 8'd220; green = 8'd30; blue = 8'd30; // mobil: merah
    end else if (sky_area) begin
        red = 8'd110; green = 8'd180; blue = 8'd240; // langit: biru
    end else if (road_area) begin
        red = 8'd60; green = 8'd60; blue = 8'd60; // jalan: abu-abu
    end else begin
        red = 8'd40; green = 8'd140; blue = 8'd50; // sisanya: rumput
    end
end
endmodule

```

C. Simpan sebagai **render_static.v**.

7. Menyambungkan render_static ke car_game

Buka kembali file **car_game.v**. Tiga baris *assign VGA_R/G/B* yang masih *8'h00* diganti dengan instance **render_static**:

```

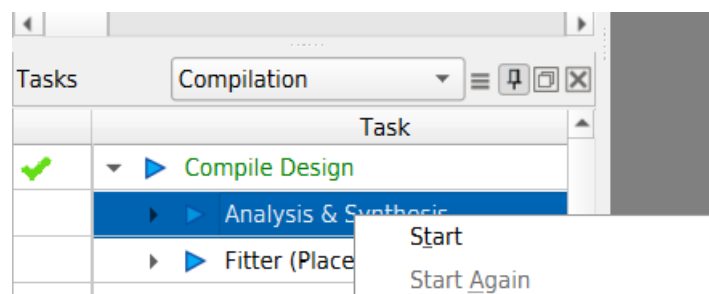
wire [7:0] r_static, g_static, b_static;
render_static render_unit (
    .clk      (pixel_clk),
    .video_on (video_on),
    .x        (x),
    .y        (y),
    .red      (r_static),
    .green    (g_static),
    .blue     (b_static)
);

assign VGA_R = r_static;
assign VGA_G = g_static;
assign VGA_B = b_static;

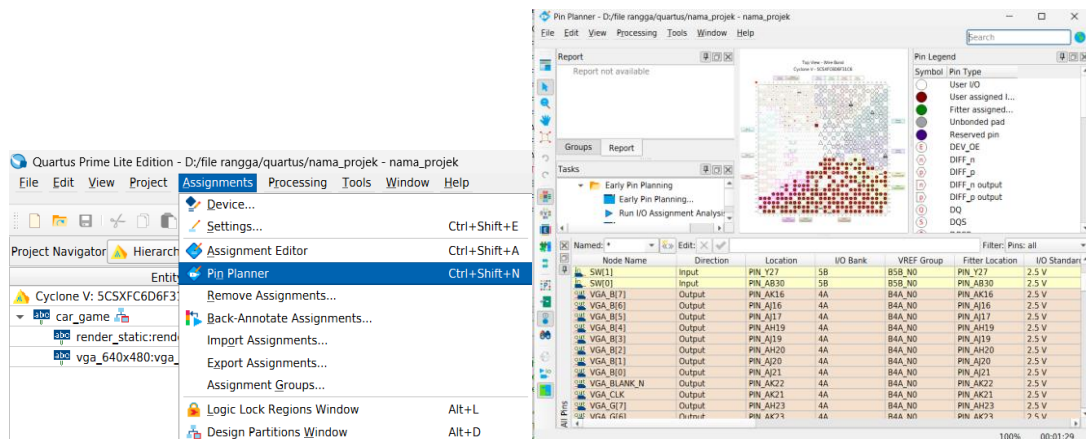
```

8. Kompilasi, Pengaturan Pin, dan Uji Coba di Board FPGA

A. Masuk ke bagian **Task >> Analysis & Synthesis >> klik-kanan >> Start**



B. Masuk ke bagian **Pin Planner (Assignments >> Pin Planner)**



C. Masukkan pin berikut pada bagian tabel Location:

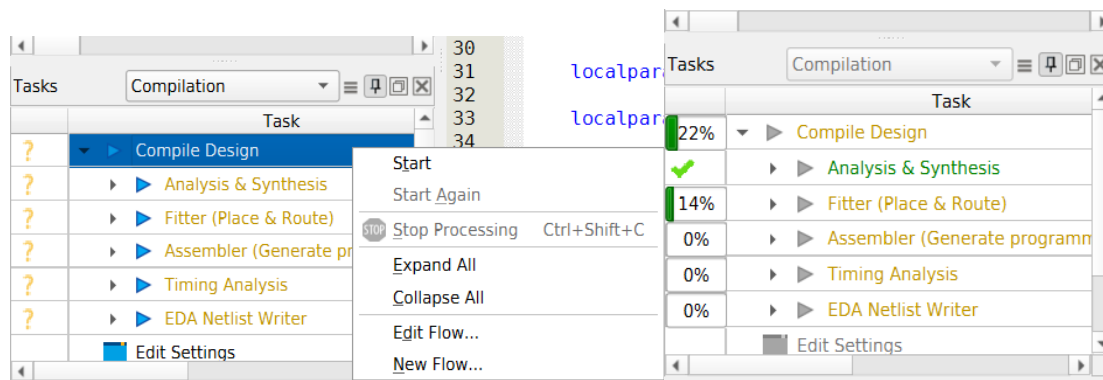
Node Name	Location
CLOCK_50	PIN_AF14
SW[9]	PIN_AA30
VGA_CLK	PIN_AK21
VGA_HS	PIN_AK19
VGA_VS	PIN_AK18
VGA_BLANK_N	PIN_AK22
VGA_SYNC_N	PIN_AJ22
VGA_R[7]	PIN_AJ26
VGA_R[6]	PIN_AG26
VGA_R[5]	PIN_AF26
VGA_R[4]	PIN_AH27
VGA_R[3]	PIN_AJ27
VGA_R[2]	PIN_AK27
VGA_R[1]	PIN_AK28
VGA_R[0]	PIN_AK29
VGA_G[7]	PIN_AH23
VGA_G[6]	PIN_AK23
VGA_G[5]	PIN_AH24
VGA_G[4]	PIN_AJ24
VGA_G[3]	PIN_AK24
VGA_G[2]	PIN_AH25
VGA_G[1]	PIN_AJ25

Node Name	Location
VGA_G[0]	PIN_AK26
VGA_B[7]	PIN_AK16
VGA_B[6]	PIN_AJ16
VGA_B[5]	PIN_AJ17
VGA_B[4]	PIN_AH19
VGA_B[3]	PIN_AJ19
VGA_B[2]	PIN_AH20
VGA_B[1]	PIN_AJ20
VGA_B[0]	PIN_AJ21

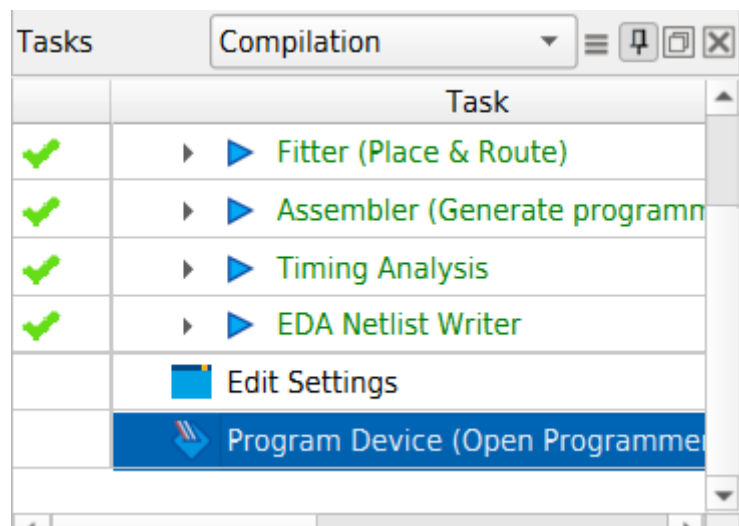
Hanya SW[9] yang perlu diberi pin, karena hanya bit tersebut yang dipakai (sebagai reset) pada tahap ini; SW[0] hingga SW[8] belum digunakan sehingga tidak perlu di-assign.

Lokasi pin di atas berlaku untuk DE10-Standard. Kalau board Anda beda revisi/varian, cocokkan dulu dengan User Manual board masing-masing sebelum kompilasi.

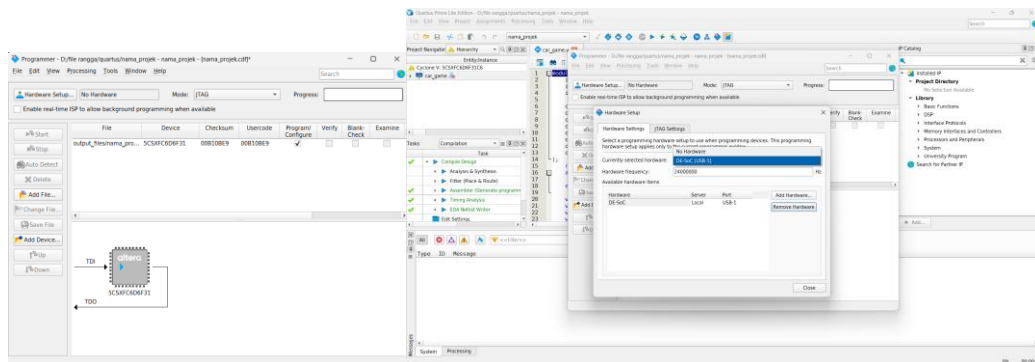
D. Kompilasi keseluruhan Task (Task >> Compile Design >> klik kanan >> Start), tunggu selesai



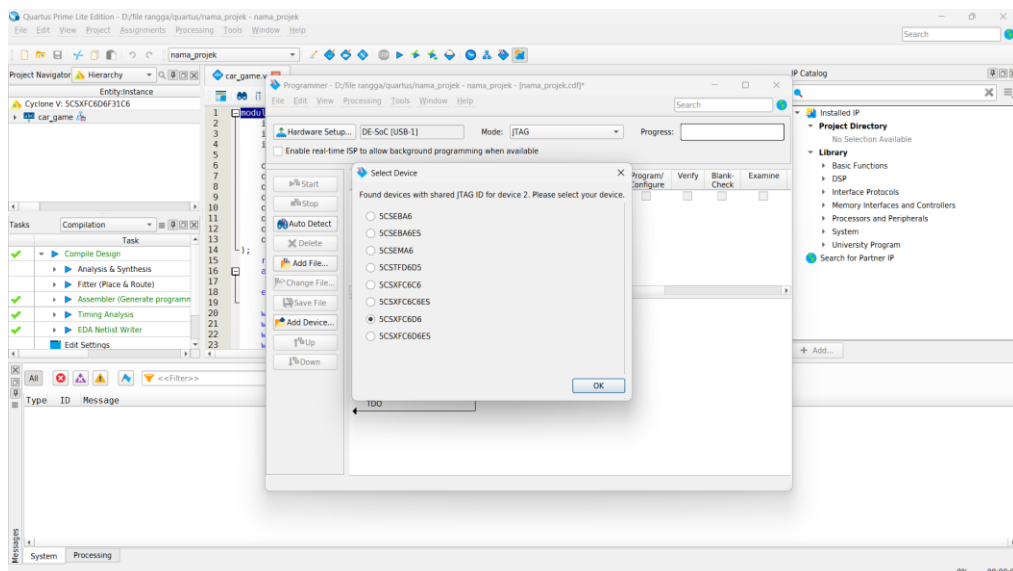
E. Setelah selesai kompilasi, klik Program Device (Open Programmer)



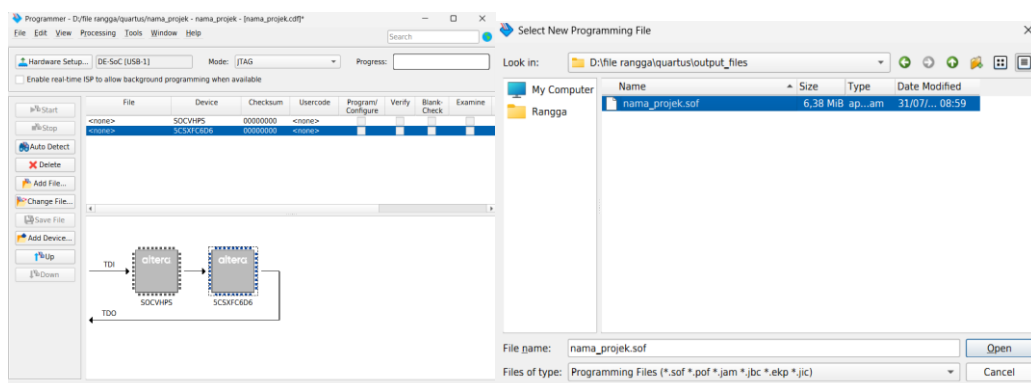
F. Setelah muncul jendela Programmer, hubungkan port USB-Blaster board FPGA ke PC/Laptop, dan nyalakan board FPGA-nya. Klik di **Hardware Setup...** >> pilih **DE-SoC** di bagian kolom hardware >> **Close**



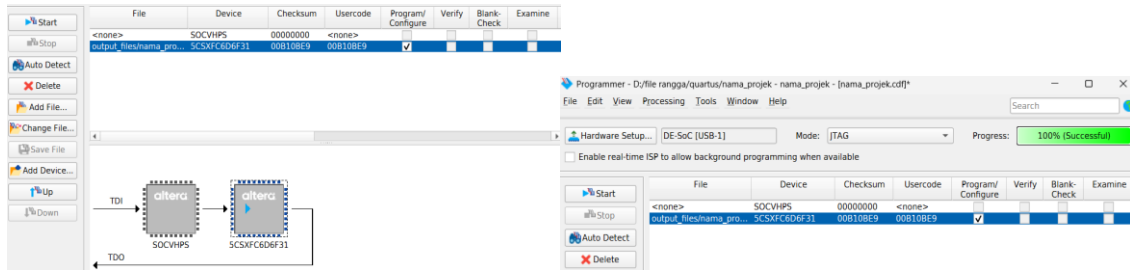
G. Kembali ke jendela Programmer lagi, pilih **Auto Detect** >> **5CSXFC6D6**, jika muncul peringatan, klik **Yes**



H. Selanjutnya pada device **5CSXFC6D6**, klik **<none>** di kolom File lalu arahkan ke file ***.sof** yang ada di folder **output_files** di folder proyek Anda >> **Open**



I. Centang kolom **Program/Configure** di baris file ***.sof** yang baru dipilih, lalu klik **Start**



J. Jika progress sudah **successful**, maka monitor akan menampilkan desain berikut:



K. FPGA di board Anda sudah terisi file desain. Jika ingin menambah/mengubah elemen lain, Anda bisa menambahnya/mengubahnya di file render (module **render_static**).

Troubleshooting Singkat

Gejala	Kemungkinan Penyebab & Solusi
Layar tetap hitam/blank	Cek SW[9] tidak dalam posisi ON (reset aktif menahan tampilan); cek pin VGA_CLK/HS/VS/BLANK_N sudah benar semua.
Error "module not found" saat compile	File .v yang baru dibuat belum ter-add ke proyek. Cek Project Navigator, atau tambahkan manual lewat Project >> Add/Remove Files in Project.
Warning "port not used" untuk KEY atau clk	Aman diabaikan pada tahap ini, kedua port memang belum dipakai, akan disambungkan di tahap pengembangan lanjutan.
Gambar muncul tapi warna/posisi salah	Cek urutan if/else if di render_static.v, ingat, kondisi paling atas yang menang kalau ada area yang tumpang tindih.
Programmer tidak mendeteksi board	Pastikan kabel USB-Blaster tercolok dan board sudah menyala, lalu klik Auto Detect ulang.

Dua bagian di bawah ini sifatnya opsional di luar 7 tahap utama di atas. Ditulis sebagai insight tambahan untuk yang ingin mengembangkan lebih lanjut: membuat jalan terlihat bergerak (motion), dan menampilkan gambar/sprite (bukan cuma bentuk geometris) di render_static.v.

Insight Tambahan: Teknik Motion

A. Kecepatan Dunia yang Mengikuti Kesulitan

Satu wire menjumlahkan kecepatan base + bonus kesulitan (naik seiring skor) + bonus sementara saat nitro aktif daripada satu SCROLL_SPEED tetap. Karena jalur, pohon, dan hambatan menggunakan jalur yang sama, semuanya otomatis mempercepat secara bersamaan tanpa perlu menyinkronkan kecepatan yang berbeda.

render_static.v (world_scroll_speed)

```
wire [9:0] world_scroll_speed = SCROLL_SPEED + obstacle_speed_bonus
                                + (nitro_active ? 10'd3 : 10'd0);

// dipakai ulang oleh seg_dist (tikungan), anim_y (marka/pohon),
// DAN obstacle_y (rintangan) satu sumber kecepatan untuk semuanya
```

B. Tikungan Otomatis yang Halus

Jalan berbelok otomatis melalui empat tahap berulang: lurus, belok kanan, belok kiri, dan lurus. Tapi curve_amount (seberapa jauh jalan bengkok) tidak langsung melompat ke nilai fase target setiap beberapa frame; sebaliknya, ia bergerak ke arah target dengan satu langkah kecil (CURVE_STEP). Ini membuat transisi terasa halus daripada patah tiba-tiba. Setiap kali satu putaran penuh selesai, tikungan menjadi lebih tajam sampai batas maksimum.

render_static.v (easing curve_amount)

```
if (curve_update_counter >= CURVE_UPDATE_DIVIDER - 1) begin
    curve_update_counter <= 0;
    if (curve_amount < target_curve)
        curve_amount <= curve_amount + CURVE_STEP;    // gerak SATU langkah
    else if (curve_amount > target_curve)
        curve_amount <= curve_amount - CURVE_STEP;    // ke arah target
end else begin
    curve_update_counter <= curve_update_counter + 1;
end
```

Target_curve memiliki tiga nilai potensial: +current_curve_max (fase belok kanan), -current_curve_max (fase belok kiri), atau 0 (fase lurus). Kurva_jumlah hanya dapat "mengejar" salah satu nilai tersebut setiap saat.

C. Setir Mobil dengan Batas yang Ikut Menyempit/Melebar Sesuai Tikungan

Selama steer_left/steer_right ditahan, mobil bergerak bebas (car_offset) dan saat dilepas, berhenti persis di posisi itu (bukan otomatis kembali ke tengah). Untuk mencegah mobil "menembus" aspal saat jalan menikung tajam, batas kiri-kanan yang dapat dicapai tidak dihitung ulang setiap frame dari tepi jalan asli, yang sudah melengkung, pada baris tersempit hitbox mobil.

render_static.v (car_offset dengan batas dinamis)

```
if (steer_left && !steer_right) begin
    if ((car_offset - CAR_STEP) < car_offset_min_final)
        car_offset <= car_offset_min_final;    // mentok, jangan lewat
    else
        car_offset <= car_offset - CAR_STEP;
end else if (steer_right && !steer_left) begin
    if ((car_offset + CAR_STEP) > car_offset_max_final)
```

```

        car_offset <= car_offset_max_final;
    else
        car_offset <= car_offset + CAR_STEP;
    end
    // car_offset_min_final/max_final dihitung dari road_left_car/
    // road_right_car tepi jalan ASLI di baris hitbox mobil,
    // bukan sekadar batas layar tetap

```

Ada pengaman tambahan (MIN_DRIVABLE_WIDTH): jika tikungan begitu tajam sehingga celah kiri-kanan jalan lebih sempit dari lebar mobil, batasnya otomatis dilebarkan kembali di sekitar titik tengahnya. Ini memastikan bahwa pemain selalu punya ruang untuk menghindari dan tidak terjebak di tempat di mana rintangan kebetulan bertabrakan dengan mereka.

D. Rintangan Mendekat Makin Cepat

Seiring dengan peningkatan skor, rintangan harus semakin dekat. Jika kita menggunakan skor / OBSTACLE_SPEEDUP_SCORE, misalnya / 50, itu akan terlihat seperti hal yang sederhana. Namun, karena skor 50 bukan pangkat 2, kita perlu menggabungkan rangkaian pembagi yang berat. Karena hasilnya hanya digunakan sampai tujuh tingkat tetap, cukup untuk membandingkannya dengan deretan if/else if yang terdiri dari tujuh batas skor tetap. Hasilnya sama dan lebih ringan di FPGA.

render_static.v (obstacle_speed_bonus tanpa pembagian)

```

wire [9:0] obstacle_speed_bonus =
    (score >= 16'd350) ? 10'd7 :
    (score >= 16'd300) ? 10'd6 :
    (score >= 16'd250) ? 10'd5 :
    (score >= 16'd200) ? 10'd4 :
    (score >= 16'd150) ? 10'd3 :
    (score >= 16'd100) ? 10'd2 :
    (score >= 16'd50)  ? 10'd1 :
                      10'd0;

```

Bonus yang sama ini juga dipakai buat CAR_STEP (kecepatan geser mobil) jadi kecepatan mobil menggeser dan kecepatan rintangan mendekat naik BARENG dengan rasio tetap, bukan rintangan makin cepat sepihak sementara mobil tetap lambat.

E. Burung & Awan yang Tetap Bergerak Walau Mobil Berhenti

Karena di-gate sengaja oleh !game_over && game_started, semua gerakan sebelumnya (A-D) berhenti total saat game_over atau sebelum start_btn ditekan. Namun, burung dan awan di langit tidak menggunakan gerbang itu secara sengaja. Sebaliknya, mereka memiliki penghitung geser sendiri, disebut sky_scroll, yang bergerak di setiap frame, apa pun kondisi permainannya. Hasilnya adalah langit tetap terlihat hidup, dan layar start dan game over tidak terasa seperti "gambar mati".

render_static.v (sky_scroll TIDAK di-gate game_started)

```

reg [11:0] sky_scroll; // TIDAK dicek !game_over && game_started
always @(posedge clk) begin
    if (reset)
        sky_scroll <= 12'd0;

```

```

    else if (x == 10'd0 && y == 10'd0) // tetap sekali per frame
        sky_scroll <= (sky_scroll >= 12'd2559) ? 12'd0 : sky_scroll + 12'd1;
    end

    wire bird_flap    = sky_scroll[4];          // 1 bit -> animasi "ngepak" gratis
    wire [9:0] bird1_x = wrap_x(BIRD1_X0 + sky_scroll);
    wire [9:0] cloud_shift = sky_scroll[11:2]; // awan geser lebih lambat dari
    burung

```

Untuk membungkus posisi x kembali ke 0 saat melewati 640 (lebar layar), Wrap_x menggunakan deretan pengurangan bertingkat daripada operator %, pola yang sama seperti trik hindari-pembagian di poin D. Selain itu, sky_scroll sengaja dibungkus tepat di 2560 (=640x4), sehingga sky_scroll>>2 (digunakan untuk awan, yang bergerak 4 kali lebih lambat dari burung) dan juga otomatis dibungkus tepat di 640 saat counter direset ke 0 sehingga tidak ada lompatan Jarak horizontal dari pusat burung, dx, digeser dengan bit. Bit ini harus lebih besar dari 1 saat mengepak dan lebih rendah dari 2 saat tidak mengepak. Ini sangat penting karena fungsi ini dievaluasi ulang di setiap pixel layar, jadi harus seringan mungkin.

F. Hitungan Jarak Tempuh (Odometer)

Dalam odometer paling sederhana, distance_traveled hanya bertambah terus setiap frame sebesar world_scroll_speed. Namun, karena mengikuti world_scroll_speed yang sama, angka juga naik lebih cepat saat nitro aktif.

render_static.v (distance_traveled)

```

reg [23:0] distance_traveled;
// tiap frame:
distance_traveled <= distance_traveled + world_scroll_speed;

```

G. Rintangan Muncul di Jalur Acak Pakai LFSR

Setiap kali respawn dari obstacle_rand, lajur rintangan (kiri, tengah, atau kanan) dan jenis rintangan (motor atau mobil) di-roll ulang, sebuah LFSR 8-bit (register geser dengan feedback). Ini adalah metode murah untuk membuat urutan bit yang "acak" tanpa menggunakan generator random sungguhan. Ditambahkan OBSTACLE_SPAWN_GAP untuk membuat jeda ~0,8 detik antara rintangan untuk mencegah penumpukan mepet berulang.

render_static.v (obstacle_rand & spawn gap)

```

reg [7:0] obstacle_rand; // LFSR sumber acak murah
reg [1:0] obstacle_lane; // 0=kiri, 1=tengah, 2=kanan, di-roll saat respawn
localparam [15:0] OBSTACLE_SPAWN_GAP = 16'd50; // jeda ~0.8 detik sebelum respawn

```

H. Bintang Nyawa & Nitro

Bintang pickup menggunakan mesin gerak yang SAMA dengan rintangan (posisi, lajur, perspektif, dan kecepatan), hanya digambar dengan cara yang berbeda (kilau emas, bukan drum bahaya), dan efeknya kebalikan: menambah nyawa atau bonus_is_nitro=1 menambah +3 ke kecepatan dunia selama frame NITRO_DURATION.

render_static.v (nitro_timer & bonus_is_nitro)

```
reg bonus_is_nitro;          // di-roll saat spawn: nitro atau bintang nyawa biasa
reg [15:0] nitro_timer;
wire nitro_active = (nitro_timer != 16'd0); // dipakai balik di
world_scroll_speed
```

I. Mobil Rival/Lalu Lintas yang Bisa Disalip

rival_y bergerak dengan matematika perspektif yang sama dengan rintangan, tetapi kecepatannya hanya separuh dari world_scroll_speed. Ini membuatnya dianggap sebagai "mobil lain yang ikut melaju" (yang dapat disalip untuk bonus poin), bukan rintangan diam yang mengancam.

render_static.v (rival_y, setengah kecepatan dunia)

```
// rival bergerak SEPARUH kecepatan dunia itu yang bikin terlihat
// seperti kendaraan lain, bukan rintangan diam
rival_y <= rival_y + (world_scroll_speed >> 1);
```

J. Efek Tabrakan: Layar Kilat Merah dan Guncangan

Saat terjadi benturan, hit_flash mengisi nilai maksimum dan kemudian mengurangi satu tiap frame, menghasilkan flash merah yang cepat pudar. Selain itu, shake_timer memicu shake_offset_x/y, yang merupakan pergeseran kecil posisi gambar yang arah/besarnya diambil dari obstacle_rand bit-bit, yang digunakan ulang LFSR yang sama. Dengan demikian, guncangan terasa acak tanpa menggunakan generator acak yang berbeda.

render_static.v (hit_flash & guncangan layar)

```
hit_flash <= 4'd12; // lalu berkurang 1 tiap frame -> flash yang meluruh
shake_timer <= 4'd10;
shake_offset_x <= (obstacle_rand[1] ? 1 : -1) * shake_timer[3:2];
shake_offset_y <= (obstacle_rand[3] ? 1 : -1) * shake_timer[3:2];
```

K. Nyaris Tabrakan: Flash Emas dan Combo Beruntun

nm_flash (near-miss flash) menyala emas sesaat setiap kali mobil lewat sangat dekat dengan rintangan tanpa menabraknya. Combo_count bertambah tiap nyaris-tabrakan berturut-turut, tetapi direset ke 0 begitu satu rintangan tertabrak. Ini mendorong pemain untuk mepet lebih jauh daripada hanya menghindar aman.

render_static.v (nm_flash & combo_count)

```
nm_flash <= 4'd12;
combo_count <= combo_count + 8'd1; // reset ke 0 saat benar-benar tertabrak
```

L. Perayaan Skor: Confetti Tiap Melewati Ambang Tertentu

Milestone_target menyimpan skor berikutnya yang harus dilewati untuk merayakan, misalnya tiap kelipatan MILESTONE_STEP. Saat skor menyentuhnya, celebrate_timer diisi penuh (memicu confetti selama beberapa frame), dan milestone_target langsung dinaikkan ke ambang berikutnya.

render_static.v (celebrate_timer & milestone_target)

```

if (score >= milestone_target) begin
    celebrate_timer <= 5'd28;           // confetti tampil ~28 frame
    milestone_target <= milestone_target + MILESTONE_STEP;
end

```

M. Cuaca yang Bergerak: Hujan, Salju, dan Petir

Untuk menggerakkan garis hujan atau butiran salju turun, rain_scroll dan snow_scroll masing-masing memiliki kecepatan jatuh tertentu (hujan lebih cepat dari salju), sedangkan lightning_flash memberi kilatan putih-kebiruan sesaat yang munculnya diberi jeda acak (lightning_cooldown, di-reset pakai obstacle_rand lagi) supaya petir tidak menyambar dengan pola yang dapat ditebak.

render_static.v (rain_scroll, snow_scroll, lightning_flash)

```

rain_scroll <= rain_scroll + 10'd10;    // hujan turun lebih cepat
snow_scroll <= snow_scroll + 10'd3;     // salju turun lebih lambat
lightning_cooldown <= obstacle_rand + 10'd120; // jeda petir berikutnya, acak

```

N. Kincir Angin Berputar

Bukan animasi berputar asli. Windmill_spin hanyalah sebagian yang dibalik (putar) di tiap frame Windmill_Spin_Period dan digunakan untuk memilih salah satu dari dua orientasi baling-baling yang telah digambar sebelumnya. Toggle sederhana ini cukup untuk memberikan kesan "berputar" dari jarak jauh tanpa perlu menghitung sudut rotasi sebenarnya.

render_static.v (windmill_spin, toggle 1 bit)

```

reg windmill_spin;
if (windmill_spin_counter >= WINDMILL_SPIN_PERIOD - 1) begin
    windmill_spin_counter <= 0;
    windmill_spin <= ~windmill_spin; // cuma membalik 1 bit
end

```

O. Kelap-Kelip Emas pada Highscore

Shimmer_phase digunakan sebagai komponen warna yang berkilauan dan naik lambat (dikontrol oleh shimmer_counter sebagai pembagi frame). Ini memberikan efek "berkilau" pada digit highscore saat pemain hampir mengalahkan rekornya sendiri, tanpa perlu tabel warna terpisah.

render_static.v (shimmer_phase)

```

if (shimmer_counter >= SHIMMER_DIVIDER - 1) begin
    shimmer_counter <= 0;
    shimmer_phase <= shimmer_phase + 1; // dipakai jadi komponen warna berkilauan
end

```

P. Transisi Fade Layar

Per frame menuju target, fade_level naik/turun sebesar FADE_STEP secara bertahap. Ini dapat digunakan dengan fungsi blend4 untuk meredupkan seluruh warna scene secara halus, bukan loncat gelap-terang instan.

render_static.v (fade_level, transisi halus)

```

if (fade_target > fade_level)
    fade_level <= fade_level + FADE_STEP;
else if (fade_target < fade_level)
    fade_level <= fade_level - FADE_STEP;
// dipakai: red = blend4(red, 0, fade_level);

```

Q. Hitung Mundur Mulai: 3-2-1-GO!

Setelah menekan start_btn, countdown_active menyala, dan semua logic dunia yang di-gate!game_over && game_started tetap beku. Countdown_num turun 3,2,1 tiap COUNTDOWN_FRAMES_PER_NUM frame, dan game_started baru diset 1 tepat saat hitungan selesai, diikuti oleh go_banner_timer yang memicu tulisan "GO!" berkedip sesaat.

render_static.v (countdown 3-2-1-GO)

```

if (countdown_frame_counter >= COUNTDOWN_FRAMES_PER_NUM - 1) begin
    if (countdown_num <= 1) begin
        countdown_active <= 0;
        game_started <= 1;           // dunia baru mulai gerak DI SINI
        go_banner_timer <= GO_BANNER_FRAMES;
    end else
        countdown_num <= countdown_num - 1;
end

```

Di hampir semua poin A-Q di atas, pola ini berulang: satu register dicek atau diupdate setiap frame (menggunakan x=0 dan y=0), kemudian jalur lain membaca register tersebut untuk menentukan warna atau posisi pixel saat ini. Setelah pola ini dikuasai dalam dua contoh, seperti menggunakan scroll_y untuk jalan, menambahkan animasi tambahan, seperti cuaca, power-up, dan efek layar, sebenarnya mengulang pola yang sama persis, hanya dengan cara penghitungan yang berbeda dan bagaimana hasilnya digunakan untuk gambar.

Insight Tambahan: Menambahkan Gambar (Sprite) via Memory Initialization File (.mif)

Sejauh ini seluruh bentuk (jalan, mobil, garis) digambar murni lewat perbandingan koordinat (x, y) di dalam kode Verilog. cara ini cocok untuk bentuk-bentuk geometris sederhana, tapi tidak praktis untuk menampilkan gambar/logo/sprite yang detail. Untuk itu, Verilog/Quartus menyediakan cara menyimpan data gambar sebagai isi memori (ROM) lewat file .mif (Memory Initialization File), lalu membacanya per-pixel saat digambar.

A. Siapkan data gambar dalam format **.mif**. File ini berisi nilai warna tiap pixel gambar, disusun berurutan sebagai isi memori. Contoh cuplikan format file **mobil.mif** untuk gambar 8x8 pixel (1 byte per pixel, format grayscale sederhana):

mobil.mif (contoh, gambar 8x8 pixel)

```

WIDTH=8;
DEPTH=64;
ADDRESS_RADIX=UNS;
DATA_RADIX=HEX;

```

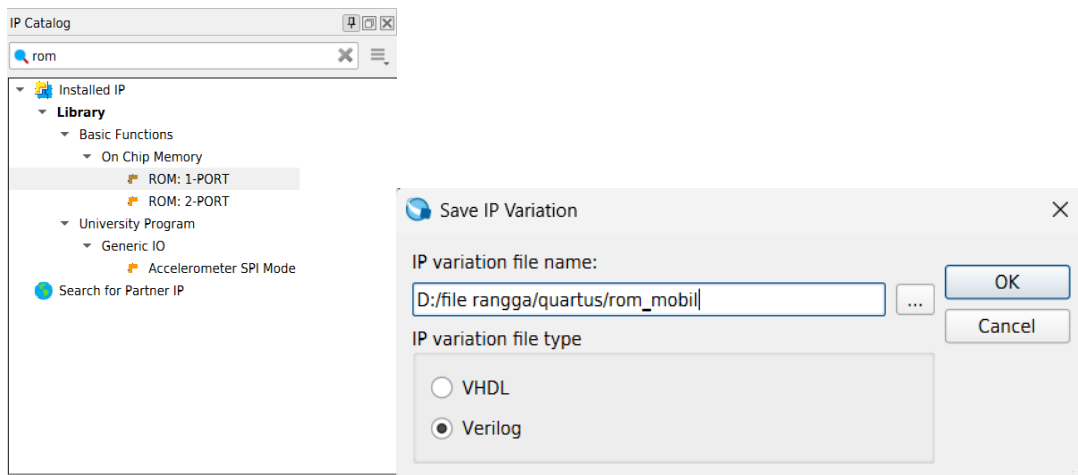
```

CONTENT BEGIN
  0 : FF;
  1 : FF;
  2 : 00;
  3 : 00;
  ... isi lanjutan sesuai jumlah pixel (DEPTH) ...
  63 : FF;
END;

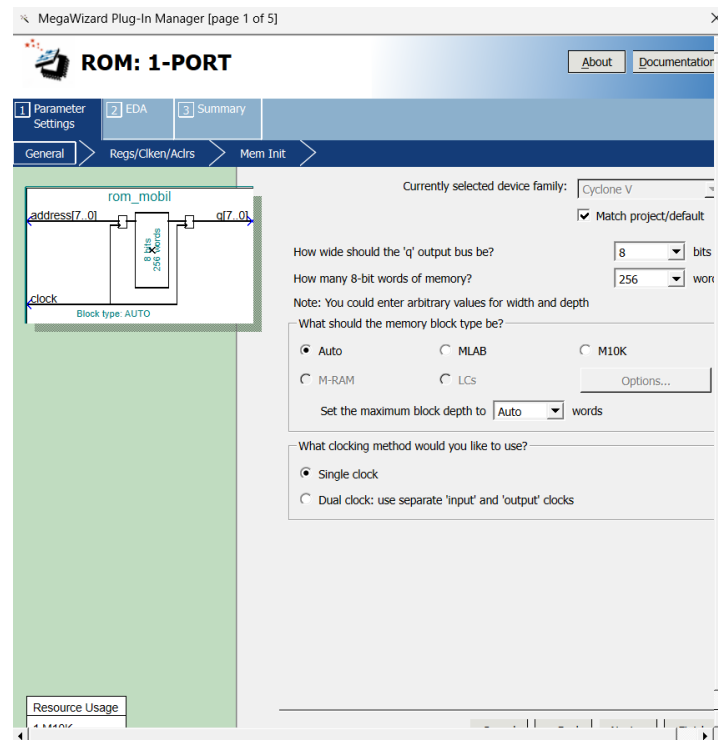
```

File **.mif** seperti ini biasanya tidak ditulis tangan pixel-per-pixel, melainkan dihasilkan dari gambar asli (PNG/BMP) lewat script konversi (Python/MATLAB) yang membaca nilai warna tiap pixel lalu menuliskannya ke format **.mif** di atas.

B. Tambahkan komponen ROM lewat **IP Catalog: Tools >> IP Catalog >>** cari "**ROM: 1-PORT**" >> pilih **Memory Compiler >> On-Chip Memory (ROM)**. Pada langkah selanjutnya masukkan Nama **IP Core** pada jendela "**save IP Variation**" >> klik **OK**



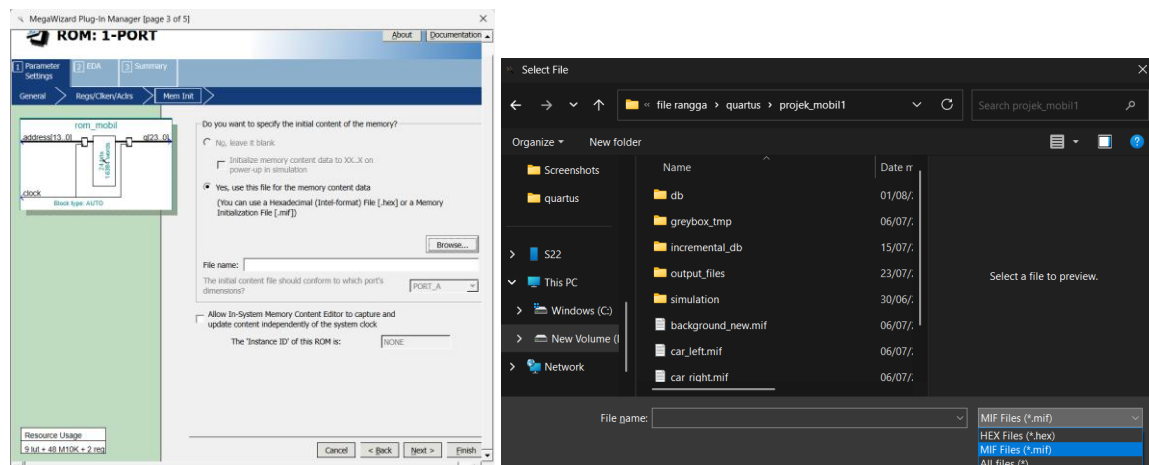
C. Setelah klik **OK**, akan terbuka jendela baru bernama **MegaWizard** atau **IP Parameter Editor**. Di sini anda wajib menyesuaikan **width** dan **depth** dengan gambar **.mif**.



Catatan: Width ('q' output bus): Ini adalah jumlah bit warna per piksel. (Contoh: isi 8 untuk 8-bit, atau 12 untuk warna VGA 12-bit).

Depth (How many words): Ini adalah total jumlah piksel gambarmu. (Contoh: jika gambarmu resolusinya 50x50 piksel, maka isikan 2500).

D. Setelah memasukkan **Width** dan **Depth**-nya, klik **Next** sampai page 3. Di page 3 kalian klik **Browse...** >> Pilih terlebih dahulu File yang mau digunakan (**.mif**) >> **Finish**. Setelah itu akan muncul File ROM yang sudah anda buat tadi



E. Sambungkan modul ROM tersebut ke render_static.v: **alamat (address)** diisi dari posisi pixel relatif terhadap gambar, lalu data keluarannya (**q**) dipakai sebagai warna:

render_static.v (menampilkan gambar dari ROM)

```
// posisi gambar di layar (misal 8x8 pixel, pojok kiri-atas di 300,50)
localparam IMG_X = 10'd300;
localparam IMG_Y = 10'd50;
localparam IMG_W = 10'd8;
```

```

localparam IMG_H = 10'd8;

wire img_area = (x >= IMG_X) && (x < IMG_X + IMG_W) &&
                (y >= IMG_Y) && (y < IMG_Y + IMG_H);

// alamat ROM = posisi pixel relatif di dalam gambar (baris * lebar + kolom)
wire [5:0] img_addr = (y - IMG_Y) * IMG_W + (x - IMG_X);
wire [7:0] img_pixel;

logo_rom logo_rom_unit ( //Jangan Lupa ganti dengan nama file ROM-nya
    .address (img_addr),
    .clock   (clk),
    .q       (img_pixel)
);

// di dalam always @(*), tambahkan cabang baru SEBELUM car_area:
// end else if (img_area) begin
//     red = img_pixel; green = img_pixel; blue = img_pixel;

```

Catatan: file .mif dan file rom yang sebelumnya di generate harus dalam satu folder yang sama. Jika berbeda bisa menyebabkan error.

F. Untuk contoh tampilan di monitor nanti akan seperti ini:

